

Part 4

Effects and I/O

Functional programming is a complete programming paradigm. All programs that we can imagine can be expressed functionally, including those that mutate data in place and interact with the external world by writing to files or reading from databases. In this part, we'll apply what we covered in parts 1–3 of this book to show how FP can express these effectful programs.

We'll begin in the next chapter by examining the most straightforward handling of external effects, using an I/O monad. This is a simplistic embedding of an imperative programming language into a functional language. The same general approach can be used for handling local effects and mutation, which we'll introduce in chapter 14. Both of these chapters will motivate the development of more composable ways to deal with effects. In chapter 15, our final chapter, we'll develop a library for streaming I/O, and discuss how to write compositional and modular programs that incrementally process I/O streams.

Our goal in this part of the book is not to cover absolutely every technique relevant to handling I/O and mutation, but to introduce the essential ideas and equip you with a conceptual framework for future learning. You'll undoubtedly encounter problems that don't look exactly like those discussed here. But along with parts 1–3, after finishing this part you'll be in good position to apply FP to whatever programming tasks you may face.

